

ELOQUENT

JAVASCRIPT 4TH EDITION

Marijn Haverbeke

Copyright © 2024 by Marijn Haverbeke

This work is licensed under a Creative Commons
attribution-noncommercial license

(<http://creativecommons.org/licenses/by-nc/3.0/>). All code in the book may
also be considered licensed under an MIT license (<https://eloquentjavascript.net/code/LICENSE>).

The illustrations are contributed by various artists: Cover by
Péchané Sumi-e. Chapter illustrations by Madalina Tantareanu.
Pixel art in Chapters 7 and 16 by Antonio Perdomo Pastor. Regular
expression diagrams in Chapter 9 generated with regexper.com by
Jeff Avallone. Game concept for Chapter 16 by Thomas Palef.

You can buy a print version of this book, with an extra bonus
chapter included, printed by No Starch Press at
[http://a-fwd.com/com=marijhaver 20&asin-com=1593279507](http://a-fwd.com/com=marijhaver20&asin-com=1593279507).

CONTENTS

INTRODUCTION	1	On programming	
	2	Why language matters	4
		What is JavaScript?	8
		Code, and what to do with it	11
		Overview of this book	
		Typographic conventions	12
			13
1 VALUES, TYPES, AND OPERATORS	15	Values	
		Numbers	16
		Strings	17
		Unary operators	20
		Boolean values	23
		Empty values	24
		Automatic type conversion	27
		Summary	27
			31

2 PROGRAM STRUCTURE 32 Expressions and statements

. 32 Bindings

. . 34 Binding names 36 The environment 37 Functions

. 38 The console.log function

. 39 Return values

. 39 Control flow

40 Conditional execution 41 while and do loops 43 Indenting Code

. 46 for loops

. 47 Breaking Out of a Loop

. 49 Updating bindings succinctly

. 50 Dispatching on a value with switch 50 Capitalization 52 Comments

. 53 Summary

. 54 Exercises

. 54

3 FUNCTIONS 58 Defining a function

. 59 Bindings and scopes 60 Nested scope 62 Functions as values 63 Declaration notation 64

Arrow functions 65 The call stack 66 Optional Arguments

. 68 Closure

. 70 Recursion

.	72 Growing functions	76
Functions and side effects		80
Summary	81 Exercises . .	
.	82	
4 DATA STRUCTURES: OBJECTS AND ARRAYS	84 The weresquirrel	
.	85 Datasets	
.	86 Properties	
.	87 Methods	88
Objects	90 Mutability . .	
.	94 The lycanthrope's log	
.	96 Computing correlation	
.	99 Array loops	
. 101 The final analysis		102
Further arrayology	105 Strings	
and their properties	107 Rest	
parameters	109 The Math	
object	111 Deconstructing	
.	113 Optional property access	
.	115	
iv		
JSON	116 Summary . .	
.	117 Exercises	
.	118	
5 HIGHER-ORDER FUNCTIONS	122 Abstraction	
.	123 Abstracting repetition	
.	124 Higher-order functions	
126 Script dataset		128
Filtering arrays		130
Transforming with map		131
Summarizing with reduce		132
Composability	134 Strings	
and character codes	136 Recognizing	

text 139 Summary
141 Exercises
142

6 THE SECRET LIFE OF OBJECTS 144 Abstract Data Types
144 Methods
146 Prototypes
148 Classes 150 Private
Properties 153 Overriding
derived properties 155 Maps
157 Polymorphism
159

v

Getters, setters, and statics 161
Symbols 163 The
iterator interface 165 Inheritance .
168 The instanceof operator
170 Summary
171 Exercises
172

7 PROJECT: A ROBOT 175 Meadowfield
175 The task
177 Persistent data 180
Simulation 182 The mail
truck’s route 185 Pathfinding
186 Exercises
189

8 BUGS AND ERRORS 191 Language
191 Strict mode
192 Types 194 Testing
196 Debugging
197 Error propagation

.....	200 Exceptions	
.....	202 Cleaning up after exceptions	
..	204 Selective catching	207

vi

Assertions	210 Summary ..
.....	211 Exercises
.....	212

9 REGULAR EXPRESSIONS	214	Creating a regular expression	
.....	215	Testing for matches	
.....	216	Sets of characters	
..	216	International characters	218
Repeating parts of a pattern			220
Grouping subexpressions			221
Matches and groups			222
Date class			224
and look-ahead			226
.....			227
.....			228
.....			229
.....			232
Dynamically creating RegExp objects			236
search method			237
property			237
.....			240
.....			243
.....			244
246			

vii

10 MODULES	249	Modular programs
-------------------	------------	------------------------

250 ES modules	251
Packages	254
CommonJS modules	256
Building and bundling	261
Module design	262
Summary	265
Exercises	266
11 ASYNCHRONOUS PROGRAMMING	268
Asynchronicity	269
Callbacks	271
Promises	273
Failure	276
Carla	279
Breaking In	280
Async functions	283
Generators	285
A Corvid Art Project	287
The event loop	292
Asynchronous bugs	294
Summary	296
Exercises	297
12 PROJECT: A PROGRAMMING LANGUAGE	300
Parsing	300
	viii
The evaluator	307
Special forms	309
The environment	311
Functions	314
Compilation	315
Cheating	316
Exercises	318
13 JAVASCRIPT AND THE BROWSER	321
Networks and the Internet	322
The Web	324
HTML	

. 325 HTML and JavaScript 329 In
the sandbox 330
Compatibility and the browser wars 331

14 THE DOCUMENT OBJECT MODEL 334 Document structure
. 334 Trees
. 336 The standard
. 338 Moving through the tree 339
Finding elements 341 Changing
the document 342 Creating nodes . . .
. 343 Attributes
. 346 Layout
. . . 347 Styling 350

ix

Cascading styles 352 Query
selectors 354 Positioning and
animating 356 Summary
. 359 Exercises
. 360

15 HANDLING EVENTS 363 Event handlers
. 363 Events and DOM nodes
. . 364 Event objects 366
Propagation 367 Default
actions 369 Key events
. 370 Pointer events
. 372 Scroll events
. 378 Focus events
380 Load event 381 Events
and the event loop 382 Timers
. 384 Debouncing
. 385 Summary
. 387 Exercises
388

16 PROJECT: A PLATFORM GAME	390	The game	
		391 The technology	
		392 Levels	393

x

Reading a level	394	Actors	
		396 Drawing	
		402 Motion and collision	
		409 Actor updates	
		414 Tracking keys	
417 Running the game			418
Exercises	421		

17 DRAWING ON CANVAS	424	SVG	
		425 The canvas element	
		426 Lines and surfaces	428
Paths		429 Curves	
		432 Drawing a pie chart	
		435 Text	
		437 Images	
438 Transformation			441
Storing and clearing transformations			445
to the game		447 Choosing a graphics interface	
		455 Summary	
		457 Exercises	
	458		

18 HTTP AND FORMS	461	The protocol	
	461		

Browsers and HTTP	464	Fetch	
-----------------------------	-----	-----------------	--

.....	467 HTTP sandboxing ..
.....	469 Appreciating HTTP
.....	470 Security and HTTPS
.....	471 Form fields
472 Focus	475
Disabled fields	477 The form as a whole
.....	477 Text fields
.....	479 Checkboxes and radio buttons
.....	481 Select fields
.....	483 File fields
..	485 Storing data client-side
Summary	491 Exercises ..
.....	492

19 PROJECT: A PIXEL ART EDITOR	495 Components
.....	496 The state
.....	498 DOM building
501 The canvas	502 The application
.....	505 Drawing tools ..
.....	509 Saving and loading
.....	513 Undo history
.....	517 Let's draw
519	

xii

Why is this so hard?	521
Exercises	522

20 NODE.JS	526 Background
527 The node command	527
Modules	529 Installing with NPM
.....	531 The filesystem module
.....	534 The HTTP module
.....	537 Streams
.....	539 A file server

.. 541 Summary .. 549

Exercises .. 550

21 PROJECT: SKILL-SHARING WEBSITE 552 Design ..

.. 553 Long polling ..

.. 555 HTTP interface ..

.. 556 The server .. 559

The client .. 570 Exercises ..

.. 579

EXERCISE HINTS 581 Program Structure ..

.. 581 Functions .. 583

Data Structures: Objects and Arrays .. 584

xiii

Higher-Order Functions .. 587 The

Secret Life of Objects .. 588 Project: A

Robot .. 590 Bugs and Errors ..

.. 591 Regular Expressions ..

.. 592 Modules ..

.. 593 Asynchronous Programming ..

.. 596 Project: A Programming Language ..

599 The Document Object Model .. 601

Handling Events .. 602 Project:

A Platform Game .. 604 Drawing on

Canvas .. 605 HTTP and Forms ..

.. 608 Project: A Pixel Art Editor ..

.. 610 Node.js ..

.. 614 Project: Skill-Sharing Website ..

. 616

“We think we are creating the system for our own purposes. We believe we are making it in our own image... But the computer is not really like us. It is a projection of a very slim part of ourselves: that portion devoted to logic, order, rule, and clarity.”

—Ellen Ullman, *Close to the Machine: Technophilia and Its Discontents*

INTRODUCTION

This is a book about instructing computers. Computers are about as common as screwdrivers today, but they are quite a bit more complex, and making them do what you want them to do isn’t always easy.

If the task you have for your computer is a common, well-understood one, such as showing you your email or acting like a calculator, you can open the appropriate application and get to

work. But for unique or open-ended tasks, there often is no appropriate application.

That is where programming may come in. *Programming* is the act of constructing a *program*—a set of precise instructions telling a computer what to do. Because computers are dumb, pedantic beasts, programming is fundamentally tedious and frustrating.

Fortunately, if you can get over that fact—and maybe even enjoy the rigor of thinking in terms that dumb machines can deal with—programming can be rewarding. It allows you to do things in seconds that would take *forever* by hand. It is a way to make your computer tool do things that it couldn't do before. On top of that, it makes for a wonderful game of puzzle solving and abstract thinking.

Most programming is done with programming languages. A

program 1

programming language is an artificially constructed language used to instruct computers. It is interesting that the most effective way we've found to communicate with a computer borrows so heavily from the way we communicate with each other. Like human languages, computer languages allow words and phrases to be combined in new ways, making it possible to express ever new concepts.

At one point, language-based interfaces, such as the BASIC and DOS prompts of the 1980s and 1990s, were the main method of interacting with computers. For routine computer use, these have largely been replaced with visual interfaces, which are easier to learn but offer less freedom. But if you know where to look, the languages are still there. One of them, *JavaScript*, is built into every modern web browser—and is thus available on almost every device.

This book will try to make you familiar enough with this language to do useful and amusing things with it.

ON PROGRAMMING

Besides explaining JavaScript, I will introduce the basic principles of programming. Programming, it turns out, is hard. The fundamental rules are simple and clear, but programs built on top of these rules tend to become complex enough to introduce their own rules and complexity. You're building your own maze, in a way, and you can easily get lost in it.

There will be times when reading this book feels terribly frustrating. If you are new to programming, there will be a lot of new material to

2

digest. Much of this material will then be *combined* in ways that require you to make additional connections.

It is up to you to make the necessary effort. When you are struggling to follow the book, do not jump to any conclusions about your own capabilities. You are fine—you just need to keep at it. Take a break, reread some material, and make sure you read and understand the example programs and exercises. Learning is hard work, but everything you learn is yours and will make further learning easier.

When action grows unprofitable, gather information;
when information grows unprofitable, sleep.

—Ursula K. Le Guin, *The Left Hand of Darkness*

A program is many things. It is a piece of text typed by a programmer, it is the directing force that makes the computer do what it does, it is data in the computer's memory, and, at the same time, it controls the actions performed on this memory. Analogies that try to compare programs to familiar objects tend to fall short. A superficially fitting one is to compare a program to a machine—lots

of separate parts tend to be involved, and to make the whole thing tick, we have to consider the ways in which these parts interconnect and contribute to the operation of the whole.

A computer is a physical machine that acts as a host for these immaterial machines. Computers themselves can do only stupidly straight forward things. The reason they are so useful is that they do these things at an incredibly high speed. A program can ingeniously combine

3

an enormous number of these simple actions to do very complicated things.

A program is a building of thought. It is costless to build, it is weightless, and it grows easily under our typing hands. But as a program grows, so does its complexity. The skill of programming is the skill of building programs that don't confuse the programmer. The best programs are those that manage to do something interesting while still being easy to understand.

Some programmers believe that this complexity is best managed by using only a small set of well-understood techniques in their programs. They have composed strict rules ("best practices") prescribing the form programs should have and carefully stay within their safe little zone.

This is not only boring—it is ineffective. New problems often require new solutions. The field of programming is young and still developing rapidly, and it is varied enough to have room for wildly different approaches. There are many terrible mistakes to make in program design, and you should go ahead and make them at least once so that you understand them. A sense of what a good program looks like is developed with practice, not learned from a list of rules.

WHY LANGUAGE MATTERS

In the beginning, at the birth of computing, there were no programming languages. Programs looked something like this:

```
00110001 00000000 00000000
00110001 00000001 00000001
```

4

```
00110011 00000001 00000010
01010001 00001011 00000010
00100010 00000010 00001000
01000011 00000001 00000000
01000001 00000001 00000001
00010000 00000010 00000000
01100010 00000000 00000000
```

This is a program to add the numbers from 1 to 10 together and print the result: $1 + 2 + \dots + 10 = 55$. It could run on a simple hypothetical machine. To program early computers, it was necessary to set large arrays of switches in the right position or punch holes in strips of card board and feed them to the computer. You can imagine how tedious and error prone this procedure was. Even writing simple programs required much cleverness and discipline. Complex ones were nearly inconceivable.

Of course, manually entering these arcane patterns of bits (the ones and zeros) did give the programmer a profound sense of being a mighty wizard. And that has to be worth something in terms of job satisfaction.

Each line of the previous program contains a single instruction. It could be written in English like this:

1. Store the number 0 in memory location 0.
2. Store the number 1 in memory location 1.
3. Store the value of memory location 1 in memory location 2.

4. Subtract the number 11 from the value in memory location

2.

5

5. If the value in memory location 2 is the number 0, continue with instruction 9.

6. Add the value of memory location 1 to memory location

0. 7. Add the number 1 to the value of memory location 1.

8. Continue with instruction 3.

9. Output the value of memory location 0.

Although that is already more readable than the soup of bits, it is still rather obscure. Using names instead of numbers for the instructions and memory locations helps.

```
Set "total" to 0.  
Set "count" to 1.  
[loop]  
Set "compare" to "count".  
Subtract 11 from "compare".  
If "compare" is 0, continue at [end].  
Add "count" to "total".  
Add 1 to "count".  
Continue at [loop].  
[end]  
Output "total".
```

Can you see how the program works at this point? The first two lines give two memory locations their starting values: `total` will be used to build up the result of the computation, and `count` will keep track of the number that we are currently looking at. The lines using `compare` are

probably the most confusing ones. The program wants to see whether `count` is equal to 11 to decide whether it can stop running. Because our hypothetical machine is rather primitive, it can test only whether a number is zero and make a decision based on that. It therefore uses the memory location labeled `compare` to compute the value of `count - 11` and makes a decision based on that value. The next two lines add the value of `count` to the result and increment `count` by 1 every time the program decides that `count` is not 11 yet.

Here is the same program in JavaScript:

```
let total = 0, count = 1;
while (count <= 10) {
  total += count;
  count += 1;
}
console.log(total);
// → 55
```

This version gives us a few more improvements. Most importantly, there is no need to specify the way we want the program to jump back and forth anymore—the `while` construct takes care of that. It continues executing the block (wrapped in braces) below it as long as the condition it was given holds. That condition is `count <= 10`, which means “the count is less than or equal to 10”. We no longer have to create a temporary value and compare that to zero, which was just an uninteresting detail. Part of the power of programming languages is that they can take care of uninteresting details for us.

At the end of the program, after the `while` construct has finished,

`console.log` operation is used to write out the result.

Finally, here is what the program could look like if we happened to have the convenient operations `range` and `sum` available, which respectively create a collection of numbers within a range and compute the sum of a collection of numbers:

```
console.log(sum(range(1, 10)));  
// → 55
```

The moral of this story is that the same program can be expressed in both long and short, unreadable and readable ways. The first version of the program was extremely obscure, whereas this last one is almost English: `log` the `sum` of the `range` of numbers from 1 to 10. (We will see in [later chapters](#) how to define operations like `sum` and `range`.)

A good programming language helps the programmer by allowing them to talk about the actions that the computer has to perform on a higher level. It helps omit details, provides convenient building blocks (such as `while` and `console.log`), allows you to define your own building blocks (such as `sum` and `range`), and makes those blocks easy to compose.

WHAT IS JAVASCRIPT?

JavaScript was introduced in 1995 as a way to add programs to web pages in the Netscape Navigator browser. The language has since been adopted by all other major graphical web browsers. It has made modern web applications possible—that is, applications with which you can interact directly without doing a page reload for every action. JavaScript

is also used in more traditional websites to provide various forms of interactivity and cleverness.

It is important to note that JavaScript has almost nothing to do with the programming language named Java. The similar name was inspired by marketing considerations rather than good judgment. When JavaScript was being introduced, the Java language was being heavily marketed and was gaining popularity. Someone thought it was a good idea to try to ride along on this success. Now we are stuck with the name.

After its adoption outside of Netscape, a standard document was written to describe the way the JavaScript language should work so that the various pieces of software that claimed to support JavaScript could make sure they actually provided the same language. This is called the ECMAScript standard, after the Ecma International organization that conducted the standardization. In practice, the terms ECMAScript and JavaScript can be used interchangeably—they are two names for the same language.

There are those who will say *terrible* things about JavaScript. Many of these things are true. When I was required to write something in JavaScript for the first time, I quickly came to despise it. It would accept almost anything I typed but interpret it in a way that was completely different from what I meant. This had a lot to do with the fact that I did not have a clue what I was doing, of course, but there is a real issue here: JavaScript is ridiculously liberal in what it allows. The idea behind this design was that it would make programming in JavaScript easier for beginners. In actuality, it mostly makes finding problems in

your programs harder because the system will not point them out to you.

This flexibility also has its advantages, though. It leaves room for techniques that are impossible in more rigid languages and makes for a pleasant, informal style of programming. After learning the language properly and working with it for a while, I have come to actually *like* JavaScript.

There have been several versions of JavaScript. ECMAScript version 3 was the widely supported version during JavaScript's ascent to dominance, roughly between 2000 and 2010. During this time, work was underway on an ambitious version 4, which planned a number of radical improvements and extensions to the language. Changing a living, widely used language in such a radical way turned out to be politically difficult, and work on version 4 was abandoned in 2008. A much less ambitious version 5, which made only some uncontroversial improvements, came out in 2009. In 2015, version 6 came out, a major update that included some of the ideas planned for version 4. Since then we've had new, small updates every year.

The fact that JavaScript is evolving means that browsers have to constantly keep up. If you're using an older browser, it may not support every feature. The language designers are careful to not make any changes that could break existing programs, so new browsers can still run old programs. In this book, I'm using the 2024 version of JavaScript.

Web browsers are not the only platforms on which JavaScript is used. Some databases, such as MongoDB and CouchDB, use JavaScript as

their scripting and query language. Several platforms for desktop and server programming, most notably the Node.js project (the subject of [Chapter 20](#)), provide an environment for programming JavaScript outside of the browser.

CODE, AND WHAT TO DO WITH IT

Code is the text that makes up programs. Most chapters in this book contain quite a lot of code. I believe reading code and writing code are indispensable parts of learning to program. Try to not just glance over the examples—read them attentively and understand them. This may be slow and confusing at first, but I promise that you'll quickly get the hang of it. The same goes for the exercises. Don't assume you understand them until you've actually written a working solution.

I recommend you try your solutions to exercises in an actual JavaScript interpreter. That way, you'll get immediate feedback on whether what you are doing is working, and, I hope, you'll be tempted to experiment and go beyond the exercises.

The easiest way to run the example code in the book—and to experiment with it—is to look it up in the online version of the book at <https://eloquentjavascript.net>. There, you can click any code example to edit and run it and to see the output it produces. To work on the exercises, go to <https://eloquentjavascript.net/code>, which provides starting code for each coding exercise and allows you to look at the solutions.

Running the programs defined in this book outside of the book's

website requires some care. Many examples stand on their own and should work in any JavaScript environment. But code in later chapters is often written for a specific environment (the browser or Node.js) and can run only there. In addition, many chapters define bigger programs, and the pieces of code that appear in them

depend on each other or on external files. The sandbox on the website provides links to ZIP files containing all the scripts and data files necessary to run the code for a given chapter.

OVERVIEW OF THIS BOOK

This book contains roughly three parts. The first 12 chapters discuss the JavaScript language. The next seven chapters are about web browsers and the way JavaScript is used to program them. Finally, two chapters are devoted to Node.js, another environment to program JavaScript in. There are five *project chapters* in the book that describe larger example programs to give you a taste of actual programming.

The language part of the book starts with four chapters that introduce the basic structure of the JavaScript language. They discuss **control structures** (such as the `while` word you saw in this introduction), **functions** (writing your own building blocks), and **data structures**. After these, you will be able to write basic programs. Next, Chapters 5 and 6 introduce techniques to use functions and objects to write more *abstract* code and keep complexity under control.

After a **first project chapter** that builds a crude delivery robot, the language part of the book continues with chapters on **error handling**

12

and **bug fixing**, **regular expressions** (an important tool for working with text), **modularity** (another defense against complexity), and **asynchronous programming** (dealing with events that take time). The **second project chapter**, where we implement a programming language, concludes the first part of the book.

The second part of the book, Chapters 13 to 19, describes the tools that browser JavaScript has access to. You'll learn to display things on the screen (Chapters 14 and 17), respond to user input (Chapter 15), and communicate over the network (Chapter 18).

There are again two project chapters in this part: building a [platform game](#) and a [pixel paint program](#).

[Chapter 20](#) describes Node.js, and [Chapter 21](#) builds a small website using that tool.

TYPOGRAPHIC CONVENTIONS

In this book, text written in a `monospaced` font will represent elements of programs. Sometimes these are self-sufficient fragments, and sometimes they just refer to part of a nearby program. Programs (of which you have already seen a few) are written as follows:

```
function factorial(n) {  
  if (n == 0) {  
    return 1;  
  } else {  
    return factorial(n - 1) * n;  
  }  
}
```

13

```
}
```

Sometimes, to show the output that a program produces, the expected output is written after it, with two slashes and an arrow in front.

```
console.log(factorial(8));  
// → 40320
```

Good luck!

*“Below the surface of the machine, the program moves.
Without effort, it expands and contracts. In great harmony,
electrons scatter and regroup. The forms on the monitor
are but ripples on the water. The essence stays invisibly
below.”*—Master Yuan-Ma, The Book of Programming

CHAPTER 1

VALUES, TYPES, AND OPERATORS

In the computer's world, there is only data. You can read data, modify data, create new data—but that which isn't data cannot be

mentioned. All this data is stored as long sequences of bits and is thus fundamentally alike.

Bits are any kind of two-valued things, usually described as zeros and ones. Inside the computer, they take forms such as a high or low electrical charge, a strong or weak signal, or a shiny or dull spot on the surface of a CD. Any piece of discrete information can be reduced to a sequence of zeros and ones and thus represented in bits.

For example, we can express the number 13 in bits. This works the same way as a decimal number, but instead of 10 different digits, we have only 2, and the weight of each increases by a factor of 2 from right to left. Here are the bits that make up the number 13, with the weights of the digits shown below them:

0	0	0	0	1	1	0	1
128	64	32	16	8	4	2	1

That's the binary number 00001101. Its nonzero digits stand for 8,

4, 15

and 1, and add up to 13.

VALUES

Imagine a sea of bits—an ocean of them. A typical modern computer has more than 100 billion bits in its volatile data storage (working memory). Nonvolatile storage (the hard disk or equivalent) tends to have yet a few orders of magnitude more.

To be able to work with such quantities of bits without getting lost, we separate them into chunks that represent pieces of information. In a JavaScript environment, those chunks are called *values*. Though all values are made of bits, they play different roles. Every value has a type that determines its role. Some values are numbers,

some values are pieces of text, some values are functions, and so on.

To create a value, you must merely invoke its name. This is convenient. You don't have to gather building material for your values or pay for them. You just call for one, and *whoosh*, you have it. Of course, values are not really created from thin air. Each one has to be stored somewhere, and if you want to use a gigantic number of them at the same time, you might run out of computer memory. Fortunately, this is a problem only if you need them all simultaneously. As soon as you no longer use a value, it will dissipate, leaving behind its bits to be recycled as building material for the next generation of values.

The remainder of this chapter introduces the atomic elements of JavaScript programs, that is, the simple value types and the operators that can act on such values.

16

NUMBERS

Values of the *number* type are, unsurprisingly, numeric values. In a JavaScript program, they are written as follows:

13

Using that in a program will cause the bit pattern for the number 13 to come into existence inside the computer's memory. JavaScript uses a fixed number of bits, 64 of them, to store a single number value. There are only so many patterns you can make with 64 bits, which limits the number of different numbers that can be represented. With N decimal digits, you can represent 10^N numbers. Similarly, given 64 binary digits, you can represent 2^{64} different numbers, which is about 18 quintillion (an 18 with 18 zeros after it). That's a lot.

Computer memory used to be much smaller, and people tended to use groups of 8 or 16 bits to represent their numbers. It was easy

to accidentally *overflow* such small numbers—to end up with a number that did not fit into the given number of bits. Today, even computers that fit in your pocket have plenty of memory, so you are free to use 64-bit chunks, and you need to worry about overflow only when dealing with truly astronomical numbers.

Not all whole numbers less than 18 quintillion fit in a JavaScript number, though. Those bits also store negative numbers, so one bit indicates the sign of the number. A bigger issue is representing nonwhole numbers. To do this, some of the bits are used to store the position of the decimal point. The actual maximum whole number that can be

17

stored is more in the range of 9 quadrillion (15 zeros)—which is still pleasantly huge.

Fractional numbers are written using a dot:

9.81

For very big or very small numbers, you may also use scientific notation by adding an e (for *exponent*), followed by the exponent of the number.

2.998e8

That's $2.998 \times 10^8 = 299,800,000$.

Calculations with whole numbers (also called *integers*) that are smaller than the aforementioned 9 quadrillion are guaranteed to always be precise. Unfortunately, calculations with fractional numbers are generally not. Just as π (pi) cannot be precisely expressed by a finite number of decimal digits, many numbers lose some precision when only 64 bits are available to store them. This is a shame, but it causes practical problems only in specific situations. The important thing is to be aware of it and treat fractional digital numbers as approximations, not as precise values.

ARITHMETIC

The main thing to do with numbers is arithmetic. Arithmetic operations such as addition or multiplication take two number values and produce a new number from them. Here is what they look like in JavaScript:

18

```
100 + 4 * 11
```

The `+` and `*` symbols are called *operators*. The first stands for addition and the second stands for multiplication. Putting an operator between two values will apply it to those values and produce a new value.

Does this example mean “Add 4 and 100, and multiply the result by 11”, or is the multiplication done before the adding? As you might have guessed, the multiplication happens first. As in mathematics, you can change this by wrapping the addition in parentheses.

```
(100 + 4) * 11
```

For subtraction, there is the `-` operator. Division can be done with the `/` operator.

When operators appear together without parentheses, the order in which they are applied is determined by the *precedence* of the operators. The example shows that multiplication comes before addition. The `/` operator has the same precedence as `*`. Likewise, `+` and `-` have the same precedence. When multiple operators with the same precedence appear next to each other, as in `1 - 2 + 1`, they are applied left to right: `(1 - 2) + 1`.

Don’t worry too much about these precedence rules. When in doubt, just add parentheses.

There is one more arithmetic operator, which you might not immediately recognize. The `%` symbol is used to represent the *remainder*

operation. $x \% y$ is the remainder of dividing x by y . For example, $314 \% 100$ produces 14, and $144 \% 12$ gives 0. The remainder operator's precedence is the same as that of multiplication and division. You'll

19

also often see this operator referred to as *modulo*.

SPECIAL NUMBERS

There are three special values in JavaScript that are considered numbers but don't behave like normal numbers. The first two are `Infinity` and `-Infinity`, which represent the positive and negative infinities. `Infinity`

`- 1` is still `Infinity`, and so on. Don't put too much trust in infinity based computation, though. It isn't mathematically sound, and it will quickly lead to the next special number: `NaN`.

`NaN` stands for "not a number", even though it *is* a value of the number type. You'll get this result when you, for example, try to calculate $0 / 0$ (zero divided by zero), `Infinity - Infinity`, or any number of other numeric operations that don't yield a meaningful result.

STRINGS

The next basic data type is the *string*. Strings are used to represent text. They are written by enclosing their content in quotes.

```
`Down on the sea`
```

```
"Lie on the ocean"
```

```
'Float on the ocean'
```

You can use single quotes, double quotes, or backticks to mark strings, as long as the quotes at the start and the end of the string match. You can put almost anything between quotes to have JavaScript make a string value out of it. But a few characters are

more difficult. You

20

can imagine how putting quotes between quotes might be hard, since they will look like the end of the string. *Newlines* (the characters you get when you press enter) can be included only when the string is quoted with backticks (`).

To make it possible to include such characters in a string, the following notation is used: a backslash (\) inside quoted text indicates that the character after it has a special meaning. This is called *escaping* the character. A quote that is preceded by a backslash will not end the string but be part of it. When an `n` character occurs after a backslash, it is interpreted as a newline. Similarly, a `t` after a backslash means a tab character. Take the following string:

```
"This is the first line\nAnd this is the second"
```

This is the actual text in that string:

```
This is the first line
And this is the second
```

There are, of course, situations where you want a backslash in a string to be just a backslash, not a special code. If two backslashes follow each other, they will collapse together, and only one will be left in the resulting string value. This is how the string “A *newline character is written like* `"\n"`.” can be expressed:

```
"A newline character is written like \"\\n\"."
```

Strings, too, have to be modeled as a series of bits to be able to exist inside the computer. The way JavaScript does this is based on the

Unicode standard. This standard assigns a number to virtually every character you would ever need, including characters from Greek, Arabic, Japanese, Armenian, and so on. If we have a number for every character, a string can be described by a sequence of numbers. And that's what JavaScript does.

There's a complication though: JavaScript's representation uses 16 bits per string element, which can describe up to 2^{16} different characters. However, Unicode defines more characters than that—about twice as many, at this point. So some characters, such as many emoji, take up two “character positions” in JavaScript strings. We'll come back to this in [Chapter 5](#).

Strings cannot be divided, multiplied, or subtracted. The `+` operator *can* be used on them, not to add, but to *concatenate*—to glue two strings together. The following line will produce the string “concatenate”:

```
"con" + "cat" + "e" + "nate"
```

String values have a number of associated functions (*methods*) that can be used to perform other operations on them. I'll say more about these in [Chapter 4](#).

Strings written with single or double quotes behave very much the same—the only difference lies in which type of quote you need to escape inside of them. Backtick-quoted strings, usually called *template literals*, can do a few more tricks. Apart from being able to span lines, they can also embed other values.

```
`half of 100 is ${100 / 2}`
```

When you write something inside `${}` in a template literal, its result will be computed, converted to a string, and included at that position. This example produces the string "half of 100 is 50".

UNARY OPERATORS

Not all operators are symbols. Some are written as words. One example is the `typeof` operator, which produces a string value naming the type of the value you give it.

```
console.log(typeof 4.5)
// → number
console.log(typeof "x")
// → string
```

We will use `console.log` in example code to indicate that we want to see the result of evaluating something. (More about that in the [next chapter](#).)

The other operators shown so far in this chapter all operated on two values, but `typeof` takes only one. Operators that use two values are called *binary* operators, while those that take one are called *unary* operators. The minus operator (`-`) can be used both as a binary operator and as a unary operator.

```
console.log(-(10 - 2))
// → -8
```

BOOLEAN VALUES

It is often useful to have a value that distinguishes between only two possibilities, like “yes” and “no” or “on” and “off”. For this purpose, JavaScript has a *Boolean* type, which has just two values, true and

false, written as those words.

COMPARISON

Here is one way to produce Boolean values:

```
console.log(3 > 2)
// → true
console.log(3 < 2)
// → false
```

The `>` and `<` signs are the traditional symbols for “is greater than” and “is less than”, respectively. They are binary operators. Applying them results in a Boolean value that indicates whether they hold true in this case.

Strings can be compared in the same way.

```
console.log("Aardvark" < "Zoroaster")
// → true
```

The way strings are ordered is roughly alphabetic but not really what you’d expect to see in a dictionary: uppercase letters are always “less” than lowercase ones, so `"Z" < "a"`, and nonalphabetic characters (`!`, `-`, and so on) are also included in the ordering. When comparing strings,

24

JavaScript goes over the characters from left to right, comparing the Unicode codes one by one.

Other similar operators are `>=` (greater than or equal to), `<=` (less than or equal to), `==` (equal to), and `!=` (not equal to).

```
console.log("Garnet" != "Ruby")
// → true
console.log("Pearl" == "Amethyst")
// → false
```

There is only one value in JavaScript that is not equal to itself, and

that is NaN (“not a number”).

```
console.log(NaN == NaN)
// → false
```

NaN is supposed to denote the result of a nonsensical computation, and as such, it isn’t equal to the result of any *other* nonsensical computations.

LOGICAL OPERATORS

There are also some operations that can be applied to Boolean values themselves. JavaScript supports three logical operators: *and*, *or*, and *not*. These can be used to “reason” about Booleans.

The `&&` operator represents logical *and*. It is a binary operator, and its result is true only if both the values given to it are true.

```
console.log(true && false)
// → false
console.log(true && true)
```

25

```
// → true
```

The `||` operator denotes logical *or*. It produces true if either of the values given to it is true.

```
console.log(false || true)
// → true
console.log(false || false)
// → false
```

Not is written as an exclamation mark (`!`). It is a unary operator that flips the value given to it—`!true` produces `false` and `!false` gives `true`. When mixing these Boolean operators with arithmetic and other operators, it is not always obvious when parentheses are needed. In practice, you can usually get by with knowing that of the operators we have seen so far, `||` has the lowest precedence, then comes `&&`,

then the comparison operators (>, ==, and so on), and then the rest. This order has been chosen such that, in typical expressions like the following one, as few parentheses as possible are necessary:

```
1 + 1 == 2 && 10 * 10 > 50
```

The last logical operator we will look at is not unary, not binary, but *ternary*, operating on three values. It is written with a question mark and a colon, like this:

```
console.log(true ? 1 : 2);  
// → 1  
console.log(false ? 1 : 2);  
// → 2
```

26

This one is called the *conditional* operator (or sometimes just *the ternary operator* since it is the only such operator in the language). The operator uses the value to the left of the question mark to decide which of the two other values to “pick”. If you write `a ? b : c`, the result will be `b` when `a` is true and `c` otherwise.

EMPTY VALUES

There are two special values, written `null` and `undefined`, that are used to denote the absence of a *meaningful* value. They are themselves values, but they carry no information.

Many operations in the language that don’t produce a meaningful value yield `undefined` simply because they have to yield *some* value. The difference in meaning between `undefined` and `null` is an accident of JavaScript’s design, and it doesn’t matter most of the time. In cases where you actually have to concern yourself with these values, I recommend treating them as mostly interchangeable.

AUTOMATIC TYPE CONVERSION

In the [introduction](#), I mentioned that JavaScript goes out of its way to accept almost any program you give it, even programs that do odd things. This is nicely demonstrated by the following expressions:

```
console.log(8 * null)
// → 0
console.log("5" - 1)
```

27

```
// → 4
console.log("5" + 1)
// → 51
console.log("five" * 2)
// → NaN
console.log(false == 0)
// → true
```

When an operator is applied to the “wrong” type of value, JavaScript will quietly convert that value to the type it needs, using a set of rules that often aren’t what you want or expect. This is called *type coercion*. The `null` in the first expression becomes `0` and the `"5"` in the second expression becomes `5` (from string to number). Yet in the third expression, `+` tries string concatenation before numeric addition, so the `1` is converted to `"1"` (from number to string).

When something that doesn’t map to a number in an obvious way (such as `"five"` or `undefined`) is converted to a number, you get the value `NaN`. Further arithmetic operations on `NaN` keep producing `NaN`, so if you find yourself getting one of those in an unexpected place, look for accidental type conversions.

When comparing values of the same type using the `==` operator, the outcome is easy to predict: you should get `true` when both values are the same, except in the case of `NaN`. But when the types differ, JavaScript uses a complicated and confusing set of rules to determine what to do. In most cases, it just tries to convert one of the values to the other value’s type. However, when `null` or `undefined`

occurs on either side of the operator, it produces true only if both sides are one of `null` or `undefined`.

28

```
console.log(null == undefined);  
// → true  
console.log(null == 0);  
// → false
```

That behavior is often useful. When you want to test whether a value has a real value instead of `null` or `undefined`, you can compare it to `null` with the `==` or `!=` operator.

What if you want to test whether something refers to the precise value `false`? Expressions like `0 == false` and `"" == false` are also true because of automatic type conversion. When you do *not* want any type conversions to happen, there are two additional operators: `===` and `!==`. The first tests whether a value is *precisely* equal to the other, and the second tests whether it is not precisely equal. Thus `"" === false` is false, as expected.

I recommend using the three-character comparison operators *defensively* to prevent unexpected type conversions from tripping you up. But when you're certain the types on both sides will be the same, there is no problem with using the shorter operators.

SHORT-CIRCUITING OF LOGICAL OPERATORS

The logical operators `&&` and `||` handle values of different types in a peculiar way. They will convert the value on their left side to Boolean type in order to decide what to do, but depending on the operator and the result of that conversion, they will return either the *original* left-hand value or the right-hand value.

The `||` operator, for example, will return the value to its left when

that value can be converted to true and will return the value on its right otherwise. This has the expected effect when the values are Boolean and does something analogous for values of other types.

```
console.log(null || "user")
// → user
console.log("Agnes" || "user")
// → Agnes
```

We can use this functionality as a way to fall back on a default value. If you have a value that might be empty, you can put `||` after it with a replacement value. If the initial value can be converted to false, you'll get the replacement instead. The rules for converting strings and numbers to Boolean values state that `0`, `NaN`, and the empty string (`""`) count as false, while all the other values count as true. That means `0 || -1` produces `-1`, and `"" || "!"` yields `!"`.

The `??` operator resembles `||` but returns the value on the right only if the one on the left is `null` or `undefined`, not if it is some other value that can be converted to `false`. Often, this is preferable to the behavior of `||`.

```
console.log(0 || 100);
// → 100
console.log(0 ?? 100);
// → 0
console.log(null ?? 100);
// → 100
```

The `&&` operator works similarly but the other way around. When the

value to its left is something that converts to false, it returns that value, and otherwise it returns the value on its right.

Another important property of these two operators is that the part to their right is evaluated only when necessary. In the case of `true || x`, no matter what `x` is—even if it's a piece of program that does something *terrible*—the result will be true, and `x` is never evaluated. The same goes for `false && x`, which is false and will ignore `x`. This is called *short-circuit evaluation*.

The conditional operator works in a similar way. Of the second and third values, only the one that is selected is evaluated.

SUMMARY

We looked at four types of JavaScript values in this chapter: numbers, strings, Booleans, and undefined values. Such values are created by typing in their name (`true`, `null`) or value (`13`, `"abc"`).

You can combine and transform values with operators. We saw binary operators for arithmetic (+, -, *, /, and %), string concatenation (+), comparison (==, !=, ===, !==, <, >, <=, >=), and logic (&&, ||, ??), as well as several unary operators (- to negate a number, ! to negate logically, and `typeof` to find a value's type) and a ternary operator (?:) to pick one of two values based on a third value.

This gives you enough information to use JavaScript as a pocket calculator but not much more. The [next chapter](#) will start tying these expressions together into basic programs.

*skin and they have to administer 10cc of JavaScript to get me to come back. (I respond well to toxins in the blood.)
Man, that stuff will kick the peaches right out your gills!"*

—_why, Why's (Poignant) Guide to Ruby

CHAPTER 2

PROGRAM STRUCTURE

In this chapter, we will start to do things that can actually be called *programming*. We will expand our command of the JavaScript language beyond the nouns and sentence fragments we've seen so far to the point where we can express meaningful prose.

EXPRESSIONS AND STATEMENTS

In [Chapter 1](#), we made values and applied operators to them to get new values. Creating values like this is the main substance of any JavaScript program. But that substance has to be framed in a larger structure to be useful. That's what we'll cover in this chapter.

A fragment of code that produces a value is called an *expression*. Every value that is written literally (such as `22` or `"psychoanalysis"`) is an expression. An expression between parentheses is also an expression, as is a binary operator applied to two expressions or a unary operator applied to one.

This shows part of the beauty of a language-based interface.

Expres 32

sions can contain other expressions in a way similar to how

subsentences in human languages are nested—a subsentence can contain its own sub sentences, and so on. This allows us to build expressions that describe arbitrarily complex computations.

If an expression corresponds to a sentence fragment, a JavaScript *statement* corresponds to a full sentence. A program is a list of state ments.

The simplest kind of statement is an expression with a semicolon after it. This is a program:

```
1;  
!false;
```

It is a useless program, though. An expression can be content to just produce a value, which can then be used by the enclosing code. How ever, a statement stands on its own, so if it doesn't affect the world, it's useless. It may display something on the screen, as with `console.log`, or change the state of the machine in a way that will affect the statements that come after it. These changes are called *side effects*. The state ments in the previous example just produce the values `1` and `true` and then immediately throw them away. This leaves no impression on the world at all. When you run this program, nothing observable happens.

In some cases, JavaScript allows you to omit the semicolon at the end of a statement. In other cases, it has to be there, or the next line will be treated as part of the same statement. The rules for when it can be safely omitted are somewhat complex and error prone. So in this book, every statement that needs a semicolon will always get one. I recommend you do the same, at least until you've learned more about

How does a program keep an internal state? How does it remember things? We have seen how to produce new values from old values, but this does not change the old values, and the new value must be used immediately or it will dissipate again. To catch and hold values, JavaScript provides a thing called a *binding*, or *variable*.

```
let caught = 5 * 5;
```

That gives us a second kind of statement. The special word (*keyword*) `let` indicates that this sentence is going to define a binding. It is followed by the name of the binding and, if we want to immediately give it a value, by an `=` operator and an expression.

The example creates a binding called `caught` and uses it to grab hold of the number that is produced by multiplying 5 by 5. After a binding has been defined, its name can be used as an expression. The value of such an expression is the value the binding currently holds. Here's an example:

```
let ten = 10;
console.log(ten * ten);
// → 100
```

When a binding points at a value, that does not mean it is tied to that value forever. The `=` operator can be used at any time on existing

34

bindings to disconnect them from their current value and have them point to a new one:

```
let mood = "light";
console.log(mood);
// → light
mood = "dark";
console.log(mood);
// → dark
```

You should imagine bindings as tentacles rather than boxes. They do not *contain* values; they *grasp* them—two bindings can refer to the same value. A program can access only the values to which it still has a reference. When you need to remember something, you either grow a tentacle to hold on to it or reattach one of your existing tentacles to it.

Let's look at another example. To remember the number of dollars that Luigi still owes you, you create a binding. When he pays back \$35, you give this binding a new value.

```
let luigisDebt = 140;
luigisDebt = luigisDebt - 35;
console.log(luigisDebt);
// → 105
```

When you define a binding without giving it a value, the tentacle has nothing to grasp, so it ends in thin air. If you ask for the value of an empty binding, you'll get the value `undefined`.

A single `let` statement may define multiple bindings. The definitions must be separated by commas:

35

```
let one = 1, two = 2;
console.log(one + two);
// → 3
```

The words `var` and `const` can also be used to create bindings, in a similar fashion to `let`.

```
var name = "Ayda";
const greeting = "Hello ";
console.log(greeting + name);
// → Hello Ayda
```

The first of these, `var` (short for “variable”), is the way bindings were declared in pre-2015 JavaScript, when `let` didn't exist yet. I'll get back to the precise way it differs from `let` in the [next chapter](#). For

now, remember that it mostly does the same thing, but we'll rarely use it in this book because it behaves oddly in some situations.

The word `const` stands for *constant*. It defines a constant binding, which points at the same value for as long as it lives. This is useful for bindings that just give a name to a value so that you can easily refer to it later.

BINDING NAMES

Binding names can be any sequence of one or more letters. Digits can be part of binding names—`catch22` is a valid name, for example—but the name must not start with a digit. A binding name may include dollar signs (\$) or underscores (_) but no other punctuation or special

36

characters.

Words with a special meaning, such as `let`, are *keywords*, and may not be used as binding names. There are also a number of words that are “reserved for use” in future versions of JavaScript, which also can't be used as binding names. The full list of keywords and reserved words is rather long:

```
break case catch class const continue debugger default delete do else
enum export extends false finally for function if implements import
interface in instanceof let new package private protected public
return static super switch this throw true try typeof var void while
with yield
```

Don't worry about memorizing this list. When creating a binding produces an unexpected syntax error, check whether you're trying to define a reserved word.

THE ENVIRONMENT

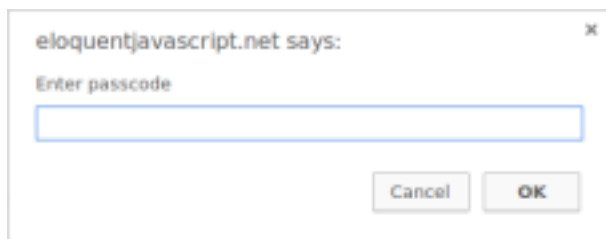
The collection of bindings and their values that exist at a given time is called the *environment*. When a program starts up, this environment is not empty. It always contains bindings that are part of the language standard, and most of the time, it also has bindings that provide ways to interact with the surrounding system. For example, in a browser, there are functions to interact with the currently loaded website and to read mouse and keyboard input.

37

FUNCTIONS

A lot of the values provided in the default environment have the type *function*. A function is a piece of program wrapped in a value. Such values can be *applied* in order to run the wrapped program. For example, in a browser environment, the binding `prompt` holds a function that shows a little dialog asking for user input. It is used like this:

```
prompt("Enter passcode");
```



Executing a function is called *invoking*, *calling*, or *applying* it. You can call a function by putting parentheses after an expression that produces a function value. Usually you'll directly use the name of the binding that holds the function. The values between the parentheses are given to the program inside the function. In the example, the `prompt` function uses the string that we give it as the text

to show in the dialog box. Values given to functions are called *arguments*. Different functions might need a different number or different types of arguments.

The `prompt` function isn't used much in modern web programming, mostly because you have no control over the way the resulting dialog

38

looks, but it can be helpful in toy programs and experiments.

THE CONSOLE.LOG FUNCTION

In the examples, I used `console.log` to output values. Most JavaScript systems (including all modern web browsers and Node.js) provide a `console.log` function that writes out its arguments to *some* text output device. In browsers, the output lands in the JavaScript console. This part of the browser interface is hidden by default, but most browsers open it when you press F12 or, on a Mac, command-option-I. If that does not work, search through the menus for an item named Developer Tools or similar.

Though binding names cannot contain period characters, `console.log` does have one. This is because `console.log` isn't a simple binding, but an expression that retrieves the `log` property from the value held by the `console` binding. We'll find out exactly what this means in [Chapter 4](#).

RETURN VALUES

Showing a dialog box or writing text to the screen is a *side effect*. Many functions are useful because of the side effects they produce. Functions may also produce values, in which case they don't need to have a side effect to be useful. For example, the function `Math.max` takes any amount of number arguments and gives back the

greatest.

```
console.log(Math.max(2, 4));  
// → 4
```

39

When a function produces a value, it is said to *return* that value. Any thing that produces a value is an expression in JavaScript, which means that function calls can be used within larger expressions. In the following code, a call to `Math.min`, which is the opposite of `Math.max`, is used as part of a plus expression:

```
console.log(Math.min(2, 4) + 100);  
// → 102
```

Chapter 3 will explain how to write your own functions.

CONTROL FLOW

When your program contains more than one statement, the statements are executed as though they were a story, from top to bottom. For example, the following program has two statements. The first asks the user for a number, and the second, which is executed after the first, shows the square of that number:

```
let theNumber = Number(prompt("Pick a number"));  
console.log("Your number is the square root of " +  
    theNumber * theNumber);
```

The function `Number` converts a value to a number. We need that conversion because the result of `prompt` is a string value, and we want a number. There are similar functions called `String` and `Boolean` that convert values to those types.

Here is the rather trivial schematic representation of straight-line control flow:

CONDITIONAL EXECUTION

Not all programs are straight roads. We may, for example, want to create a branching road where the program takes the proper branch based on the situation at hand. This is called *conditional execution*.

Conditional execution is created with the `if` keyword in JavaScript. In the simple case, we want some code to be executed if, and only if, a certain condition holds. We might, for example, want to show the square of the input only if the input is actually a number:

```
let theNumber = Number(prompt("Pick a number"));
if (!Number.isNaN(theNumber)) {
    console.log("Your number is the square root of " +
        theNumber * theNumber);
}
```

With this modification, if you enter “parrot”, no output is shown. The `if` keyword executes or skips a statement depending on the value of a Boolean expression. The deciding expression is written after the keyword, between parentheses, followed by the statement to execute. The `Number.isNaN` function is a standard JavaScript function that returns `true` only if the argument it is given is `NaN`. The `Number` function

happens to return `NaN` when you give it a string that doesn’t represent a valid number. Thus, the condition translates to “unless `theNumber` is not-a-number, do this”.

The statement after the `if` is wrapped in braces (`{` and `}`) in this example. The braces can be used to group any number of statements into a single statement, called a *block*. You could also have omitted them in this case, since they hold only a single statement, but to avoid having to think about whether they are needed, most JavaScript programmers use them in every wrapped statement like this. We'll mostly follow that convention in this book, except for the occasional one-liner.

```
if (1 + 1 == 2) console.log("It's true");  
// → It's true
```

You often won't just have code that executes when a condition holds true, but also code that handles the other case. This alternate path is represented by the second arrow in the diagram. You can use the `else` keyword, together with `if`, to create two separate, alternative execution paths:

```
let theNumber = Number(prompt("Pick a number"));  
if (!Number.isNaN(theNumber)) {  
  console.log("Your number is the square root of " +  
    theNumber * theNumber);  
} else {  
  console.log("Hey. Why didn't you give me a number?"); }  

```

If you have more than two paths to choose from, you can “chain”

mul 42

tuple `if/else` pairs together. Here's an example:

```
let num = Number(prompt("Pick a number"));  
  
if (num < 10) {  
  console.log("Small");  
}
```

```
} else if (num < 100) {  
  console.log("Medium");  
} else {  
  console.log("Large");  
}
```

The program will first check whether `num` is less than 10. If it is, it chooses that branch, shows "Small", and is done. If it isn't, it takes the `else` branch, which itself contains a second `if`. If the second condition (`< 100`) holds, that means the number is at least 10 but below 100, and "Medium" is shown. If it doesn't, the second and last `else` branch is chosen.

The schema for this program looks something like this:

WHILE AND DO LOOPS

Consider a program that outputs all even numbers from 0 to 12. One way to write this is as follows:

43

```
console.log(0);  
console.log(2);  
console.log(4);  
console.log(6);  
console.log(8);  
console.log(10);  
console.log(12);
```

That works, but the idea of writing a program is to make something *less* work, not more. If we needed all even numbers less than 1,000, this approach would be unworkable. What we need is a way to run a piece of code multiple times. This form of control flow is called a

loop.

Looping control flow allows us to go back to some point in the program where we were before and repeat it with our current program state. If we combine this with a binding that counts, we can do something like this:

```
let number = 0;
while (number <= 12) {
  console.log(number);
  number = number + 2;
}
// → 0
// → 2
// ... etcetera
```

44

A statement starting with the keyword `while` creates a loop. The word `while` is followed by an expression in parentheses and then a statement, much like `if`. The loop keeps entering that statement as long as the expression produces a value that gives `true` when converted to Boolean.

The `number` binding demonstrates the way a binding can track the progress of a program. Every time the loop repeats, `number` gets a value that is 2 more than its previous value. At the beginning of every repetition, it is compared with the number 12 to decide whether the program's work is finished.

As an example that actually does something useful, we can now write a program that calculates and shows the value of 2^{10} (2 to the 10th power). We use two bindings: one to keep track of our result and one to count how often we have multiplied this result by 2. The loop tests whether the second binding has reached 10 yet and, if

not, updates both bindings.

```
let result = 1;
let counter = 0;
while (counter < 10) {
  result = result * 2;
  counter = counter + 1;
}
console.log(result);
// → 1024
```

The counter could also have started at 1 and checked for `<= 10`, but for reasons that will become apparent in [Chapter 4](#), it is a good idea to get used to counting from 0.

45

Note that JavaScript also has an operator for exponentiation (`2 ** 10`), which you would use to compute this in real code—but that would have ruined the example.

A `do` loop is a control structure similar to a `while` loop. It differs only on one point: a `do` loop always executes its body at least once, and it starts testing whether it should stop only after that first execution. To reflect this, the test appears after the body of the loop:

```
let yourName;
do {
  yourName = prompt("Who are you?");
} while (!yourName);
console.log("Hello " + yourName);
```

This program will force you to enter a name. It will ask again and again until it gets something that is not an empty string. Applying the `!` operator will convert a value to Boolean type before negating it, and all strings except `""` convert to `true`. This means the loop continues going round until you provide a non-empty name.

INDENTING CODE

In the examples, I've been adding spaces in front of statements that are part of some larger statement. These spaces are not required—the computer will accept the program just fine without them. In fact, even the line breaks in programs are optional. You could write a program as a single long line if you felt like it.

The role of this indentation inside blocks is to make the structure

of 46

the code stand out to human readers. In code where new blocks are opened inside other blocks, it can become hard to see where one block ends and another begins. With proper indentation, the visual shape of a program corresponds to the shape of the blocks inside it. I like to use two spaces for every open block, but tastes differ—some people use four spaces, and some people use tab characters. The important thing is that each new block adds the same amount of space.

```
if (false != true) {  
  console.log("That makes sense.");  
  if (1 < 2) {  
    console.log("No surprise there.");  
  }  
}
```

Most code editor programs will help by automatically indenting new lines the proper amount.

FOR LOOPS

Many loops follow the pattern shown in the `while` examples. First a

“counter” binding is created to track the progress of the loop. Then comes a `while` loop, usually with a test expression that checks whether the counter has reached its end value. At the end of the loop body, the counter is updated to track progress.

Because this pattern is so common, JavaScript and similar languages provide a slightly shorter and more comprehensive form, the `for` loop:

47

```
for (let number = 0; number <= 12; number = number + 2) {  
  console.log(number);  
}  
// → 0  
// → 2  
// ... etcetera
```

This program is exactly equivalent to the [earlier](#) even-number-printing example. The only change is that all the statements that are related to the “state” of the loop are grouped together after `for`.

The parentheses after a `for` keyword must contain two semicolons. The part before the first semicolon *initializes* the loop, usually by defining a binding. The second part is the expression that *checks* whether the loop must continue. The final part *updates* the state of the loop after every iteration. In most cases, this is shorter and clearer than a `while` construct.

This is the code that computes 2^{10} using `for` instead of `while`:

```
let result = 1;  
for (let counter = 0; counter < 10; counter = counter + 1) { result =  
  result * 2;  
}  
console.log(result);  
// → 1024
```

BREAKING OUT OF A LOOP

Having the looping condition produce `false` is not the only way a loop can finish. The `break` statement has the effect of immediately jumping out of the enclosing loop. Its use is demonstrated in the following program, which finds the first number that is both greater than or equal to 20 and divisible by 7:

```
for (let current = 20; ; current = current + 1) {  
  if (current % 7 == 0) {  
    console.log(current);  
    break;  
  }  
}  
// → 21
```

Using the remainder (%) operator is an easy way to test whether a number is divisible by another number. If it is, the remainder of their division is zero.

The `for` construct in the example does not have a part that checks for the end of the loop. This means that the loop will never stop unless the `break` statement inside is executed.

If you were to remove that `break` statement or you accidentally write an end condition that always produces `true`, your program would get stuck in an *infinite loop*. A program stuck in an infinite loop will never finish running, which is usually a bad thing.

The `continue` keyword is similar to `break` in that it influences the progress of a loop. When `continue` is encountered in a loop body, control

jumps out of the body and continues with the loop's next iteration.

UPDATING BINDINGS SUCCINCTLY

Especially when looping, a program often needs to “update” a binding to hold a value based on that binding's previous value.

```
counter = counter + 1;
```

JavaScript provides a shortcut for this:

```
counter += 1;
```

Similar shortcuts work for many other operators, such as `result *= 2` to double `result` or `counter -= 1` to count downward. This allows us to further shorten our counting example:

```
for (let number = 0; number <= 12; number += 2) {
  console.log(number);
}
```

For `counter += 1` and `counter -= 1`, there are even shorter equivalents: `counter++` and `counter--`.

DISPATCHING ON A VALUE WITH

SWITCH It is not uncommon for code to look like

this:

```
if (x == "value1") action1();
else if (x == "value2") action2();
```

```
else if (x == "value3") action3();
else defaultAction();
```

There is a construct called `switch` that is intended to express such a “dispatch” in a more direct way. Unfortunately, the syntax JavaScript uses for this (which it inherited from the C/Java line of programming languages) is somewhat awkward—a chain of `if` statements may look better. Here is an example:

```
switch (prompt("What is the weather like?")) {
  case "rainy":
    console.log("Remember to bring an umbrella.");
    break;
  case "sunny":
    console.log("Dress lightly.");
  case "cloudy":
    console.log("Go outside.");
    break;
  default:
    console.log("Unknown weather type!");
    break;
}
```

You may put any number of `case` labels inside the block opened by `switch`. The program will start executing at the label that corresponds to the value that `switch` was given, or at `default` if no matching value is found. It will continue executing, even across other labels, until it reaches a `break` statement. In some cases, such as the “sunny” case in the example, this can be used to share some code between cases (it recommends going outside for both sunny and cloudy weather). Be

careful, though—it is easy to forget such a `break`, which will cause

the program to execute code you do not want executed.

CAPITALIZATION

Binding names may not contain spaces, yet it is often helpful to use multiple words to clearly describe what the binding represents. These are pretty much your choices for writing a binding name with several words in it:

```
fuzzylittleturtle  
fuzzy_little_turtle  
FuzzyLittleTurtle  
fuzzyLittleTurtle
```

The first style can be hard to read. I rather like the look of the underscores, though that style is a little painful to type. The standard JavaScript functions, and most JavaScript programmers, follow the final style—they capitalize every word except the first. It is not hard to get used to little things like that, and code with mixed naming styles can be jarring to read, so we follow this convention.

In a few cases, such as the `Number` function, the first letter of a binding is also capitalized. This was done to mark this function as a constructor. It will become clear what a constructor is in [Chapter 6](#). For now, the important thing is to not be bothered by this apparent lack of consistency.

COMMENTS

Often, raw code does not convey all the information you want a program to convey to human readers, or it conveys it in such a cryptic way that people might not understand it. At other times, you might just want to include some related thoughts as part of your program. This is what *comments* are for.

A comment is a piece of text that is part of a program but is completely ignored by the computer. JavaScript has two ways of writing comments. To write a single-line comment, you can use two slash characters (//) and then the comment text after it:

```
let accountBalance = calculateBalance(account);
// It's a green hollow where a river sings
accountBalance.adjust();
// Madly catching white tatters in the grass.
let report = new Report();
// Where the sun on the proud mountain rings:
addToReport(accountBalance, report);
// It's a little valley, foaming like light in a glass.
```

A // comment goes only to the end of the line. A section of text between /* and */ will be ignored in its entirety, regardless of whether it contains line breaks. This is useful for adding blocks of information about a file or a chunk of program:

```
/*
  I first found this number scrawled on the back of an old notebook. Since
  then, it has often dropped by, showing up in
```

53

```
  phone numbers and the serial numbers of products that I've bought. It
  obviously likes me, so I've decided to keep it. */
const myNumber = 11213;
```

SUMMARY

You now know that a program is built out of statements, which themselves sometimes contain more statements. Statements tend to contain expressions, which themselves can be built out of smaller expressions.

Putting statements after one another gives you a program that is executed from top to bottom. You can introduce disturbances in the

flow of control by using conditional (`if`, `else`, and `switch`) and looping (`while`, `do`, and `for`) statements.

Bindings can be used to file pieces of data under a name, and they are useful for tracking state in your program. The environment is the set of bindings that are defined. JavaScript systems always put a number of useful standard bindings into your environment.

Functions are special values that encapsulate a piece of program. You can invoke them by writing `functionName(argument1, argument2)`. Such a function call is an expression and may produce a value.

EXERCISES

If you are unsure how to test your solutions to the exercises, refer to the [introduction](#).

54

Each exercise starts with a problem description. Read this description and try to solve the exercise. If you run into problems, consider reading the hints at the [end of the book](#). You can find full solutions to the exercises online at <https://eloquentjavascript.net/code>. If you want to learn something from the exercises, I recommend looking at the solutions only after you've solved the exercise, or at least after you've attacked it long and hard enough to have a slight headache.

LOOPING A TRIANGLE

Write a loop that makes seven calls to `console.log` to output the following triangle:

```
#
##
###
####
#####
#####
```

#####

It may be useful to know that you can find the length of a string by writing `.length` after it.

```
let abc = "abc";  
console.log(abc.length);  
// → 3
```

55

FIZZBUZZ

Write a program that uses `console.log` to print all the numbers from 1 to 100, with two exceptions. For numbers divisible by 3, print "Fizz" instead of the number, and for numbers divisible by 5 (and not 3), print "Buzz" instead.

When you have that working, modify your program to print "FizzBuzz " for numbers that are divisible by both 3 and 5 (and still print "Fizz" or "Buzz" for numbers divisible by only one of those).

(This is actually an interview question that has been claimed to weed out a significant percentage of programmer candidates. So if you solved it, your labor market value just went up.)

CHESSBOARD

Write a program that creates a string that represents an 8×8 grid, using newline characters to separate lines. At each position of the grid there is either a space or a "#" character. The characters should form a chessboard.

Passing this string to `console.log` should show something like this:

```
# # # #  
# # # #  
# # # #  
# # # #
```

```
# # # #  
# # # #  
# # # #  
# # # #
```

56

When you have a program that generates this pattern, define a binding `size = 8` and change the program so that it works for any `size`, outputting a grid of the given width and height.

“People think that computer science is the art of geniuses but the actual reality is the opposite, just many people doing things that build on each other, like a wall of mini stones.”

—Donald Knuth

FUNCTIONS

Functions are one of the most central tools in JavaScript programming. The concept of wrapping a piece of program in a value has many uses. It gives us a way to structure larger programs, to reduce repetition, to associate names with subprograms, and to isolate these subprograms from each other.

The most obvious application of functions is defining new vocabulary. Creating new words in prose is usually bad style, but in programming, it is indispensable.

Typical adult English speakers have some 20,000 words in their vocabulary. Few programming languages come with 20,000 commands built in. And the vocabulary that *is* available tends to be more precisely defined, and thus less flexible, than in human language. Therefore, we *have* to introduce new words to avoid excessive verbosity.

58

DEFINING A FUNCTION

A function definition is a regular binding where the value of the binding is a function. For example, this code defines `square` to refer to a function that produces the square of a given number:

```
const square = function(x) {  
  return x * x;  
};  
  
console.log(square(12));  
// → 144
```

A function is created with an expression that starts with the keyword `function`. Functions have a set of *parameters* (in this case, only `x`) and a *body*, which contains the statements that are to be executed when the function is called. The body of a function created this way must always be wrapped in braces, even when it consists of only a single statement.

A function can have multiple parameters or no parameters at all. In the following example, `makeNoise` does not list any parameter names, whereas `roundTo` (which rounds `n` to the nearest multiple of `step`) lists two:

```
const makeNoise = function() {  
  console.log("Pling!");  
};
```

```
makeNoise();  
// → Pling!
```

59

```
const roundTo = function(n, step) {  
  let remainder = n % step;  
  return n - remainder + (remainder < step / 2 ? 0 : step); };
```

```
console.log(roundTo(23, 10));  
// → 20
```

Some functions, such as `roundTo` and `square`, produce a value, and some don't, such as `makeNoise`, whose only result is a side effect. A `return` statement determines the value the function returns. When control comes across such a statement, it immediately jumps out of the current function and gives the returned value to the code that called the function. A `return` keyword without an expression after it will cause the function to return `undefined`. Functions that don't have a

`return`

statement at all, such as `makeNoise`, similarly return `undefined`.

Parameters to a function behave like regular bindings, but their ini

tial values are given by the *caller* of the function, not the code in the function itself.

BINDINGS AND SCOPES

Each binding has a *scope*, which is the part of the program in which the binding is visible. For bindings defined outside of any function, block, or module (see [Chapter 10](#)), the scope is the whole program—you can refer to such bindings wherever you want. These are called *global*.

Bindings created for function parameters or declared inside a

function 60

can be referenced only in that function, so they are known as *local* bindings. Every time the function is called, new instances of these bindings are created. This provides some isolation between functions—each function call acts in its own little world (its local environment) and can often be understood without knowing a lot about what’s going on in the global environment.

Bindings declared with `let` and `const` are in fact local to the *block* in which they are declared, so if you create one of those inside of a loop, the code before and after the loop cannot “see” it. In pre-2015 JavaScript, only functions created new scopes, so old-style bindings, created with the `var` keyword, are visible throughout the whole function in which they appear—or throughout the global scope, if they are not in a function.

```
let x = 10; // global
if (true) {
  let y = 20; // local to block
  var z = 30; // also global
}
```

Each scope can “look out” into the scope around it, so `x` is visible inside the block in the example. The exception is when multiple bindings have the same name—in that case, code can see only the innermost one. For example, when the code inside the `halve` function refers to `n`, it is seeing its *own* `n`, not the global `n`.

```
const halve = function(n) {  
  return n / 2;  
};
```

61

```
let n = 10;  
console.log(halve(100));  
// → 50  
console.log(n);  
// → 10
```

NESTED SCOPE

JavaScript distinguishes not just global and local bindings. Blocks and functions can be created inside other blocks and functions, producing multiple degrees of locality.

For example, this function—which outputs the ingredients needed to make a batch of hummus—has another function inside it:

```
const hummus = function(factor) {  
  const ingredient = function(amount, unit, name) {  
    let ingredientAmount = amount * factor;  
    if (ingredientAmount > 1) {  
      unit += "s";  
    }  
    console.log(`${ingredientAmount} ${unit} ${name}`);  
  };  
  ingredient(1, "can", "chickpeas");  
  ingredient(0.25, "cup", "tahini");  
  ingredient(0.25, "cup", "lemon juice");  
  ingredient(1, "clove", "garlic");  
};
```

```
ingredient(2, "tablespoon", "olive oil");
```

62

```
ingredient(0.5, "teaspoon", "cumin");  
};
```

The code inside the `ingredient` function can see the `factor` binding from the outer function, but its local bindings, such as `unit` or `ingredientAmount`, are not visible in the outer function.

The set of bindings visible inside a block is determined by the place of that block in the program text. Each local scope can also see all the local scopes that contain it, and all scopes can see the global scope. This approach to binding visibility is called *lexical scoping*.

FUNCTIONS AS VALUES

A function binding usually simply acts as a name for a specific piece of the program. Such a binding is defined once and never changed. This makes it easy to confuse the function and its name.

But the two are different. A function value can do all the things that other values can do—you can use it in arbitrary expressions, not just call it. It is possible to store a function value in a new binding, pass it as an argument to a function, and so on. Similarly, a binding that holds a function is still just a regular binding and can, if not constant, be assigned a new value, like so:

```
let launchMissiles = function() {  
  missileSystem.launch("now");  
};  
if (safeMode) {  
  launchMissiles = function() {/* do nothing */};
```

```
}
```

In [Chapter 5](#), we'll discuss the interesting things that we can do by passing function values to other functions.

DECLARATION NOTATION

There is a slightly shorter way to create a function binding. When the `function` keyword is used at the start of a statement, it works differently:

```
function square(x) {  
    return x * x;  
}
```

This is a function *declaration*. The statement defines the binding `square` and points it at the given function. It is slightly easier to write and doesn't require a semicolon after the function.

There is one subtlety with this form of function definition.

```
console.log("The future says:", future());  
  
function future() {  
    return "You'll never have flying cars";  
}
```

The preceding code works, even though the function is defined *below* the code that uses it. Function declarations are not part of the regular top-to-bottom flow of control. They are conceptually moved to the top of their scope and can be used by all the code in that scope. This is